事件对象

类型：e.type

```
pointer.onclick = function( ev  ) {
            //低版本IE浏览器没有event的参数;
           console.log( window.event
            ev = ev || window.event;

        }
```

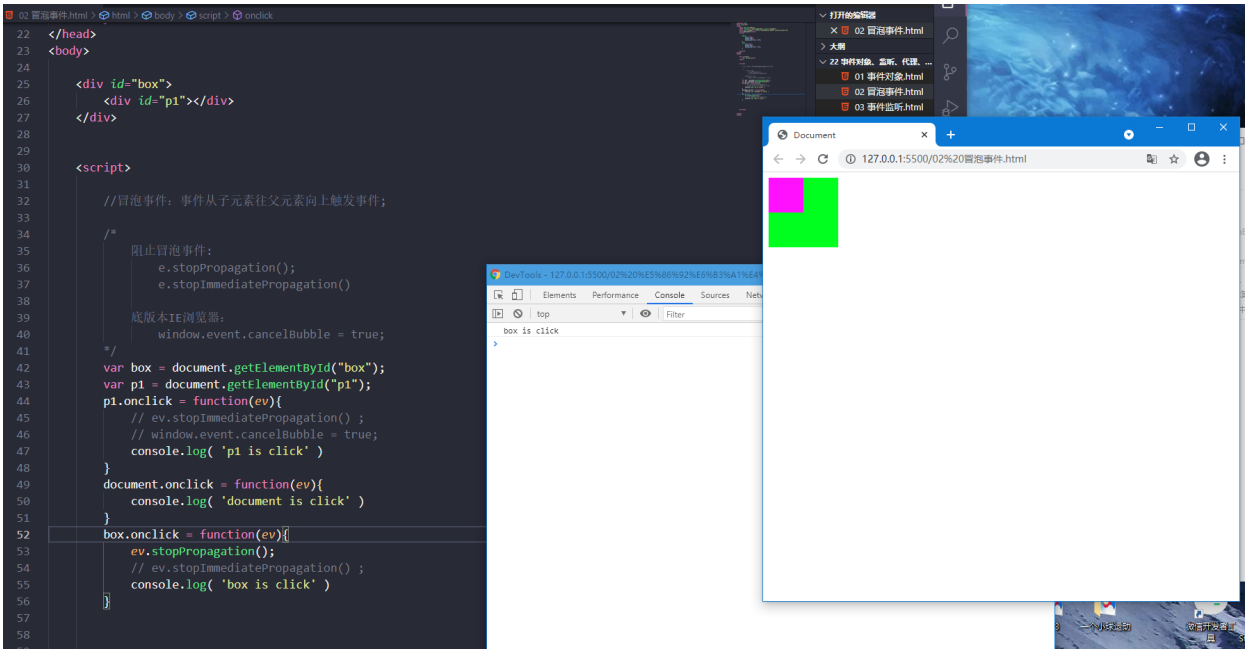
冒泡事件：

事件从子元素往父元素向上触发事件；

```
        /*
            阻止冒泡事件：
                e.stopPropagation();
                e.stopImmediatePropagation()

            底版本IE浏览器：
                window.event.cancelBubble = true;
        */
```

阻止代码放在哪里就从哪里停止冒泡事件



事件监听

//事件如果相同的时候，会出现覆盖的情况；

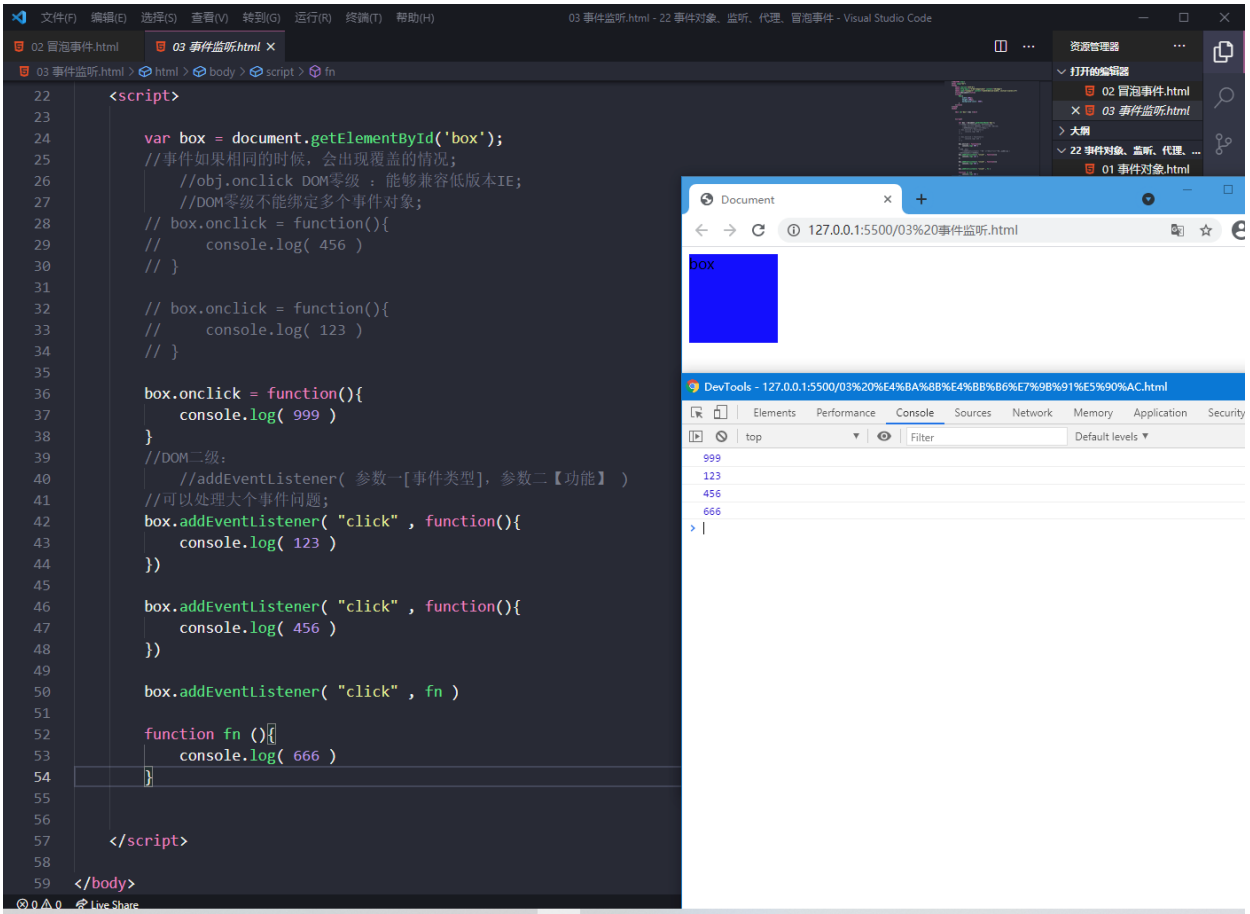
//obj.onclick DOM零级 ：能够兼容低版本IE；

//DOM零级不能绑定多个事件对象；

//DOM二级：

//addEventListener(参数一[事件类型]，参数二【功能】)

//可以处理多个事件问题；



取消监听事件

removeEventListener（）

removeAttachEvent() 低版本取消事件监听

```
<body>

    <div id='box'>
        <p id='p1'></p>
    </div>



    <script>

        var box = document.getElementById('box');
        function fn(ev){
            ev.stopPropagation();
            console.log( "box" )
        }
        box.addEventListener( "click" , fn )

        p1.addEventListener( "click" , function(ev){
            ev.stopPropagation();
            console.log( "p1" )
            //removeEventListener取消事件监听
            box.removeEventListener( "click"   , fn)

            //removeAttachEvent() 低版本取消事件监听
        })


    </script>
```

兼容低版本IE：

```
        // 如果事件监听为DOM二级的话，低版本IE需要用到attachEvent;
            //第一个参数：on + 事件类型 【click】
            //主流第一参数直接用：事件类型 ，不需要使用on的情况;
        box.attachEvent( 'onclick' , function(){
            console.log( 123 )
        } )

        box.attachEvent( 'onclick' , function(){
            console.log( 456 )
        } )

        box.attachEvent( 'onclick' , function(){
            console.log( 789 )
        } )
```

事件捕获

一.有多个参数： addEventListener(type , function(){} , boolean);

从子级到父级叫事件冒泡， 从父级到子级叫事件捕获。（ 不会冒泡 ， 当布尔值为true的时候，为事件捕获 ）

如果为false的话，不进行事件捕获， 如果为true的话，进行事件捕获。

区别：事件的触发顺序不同 IE刚开始实现的是冒泡，捕获是谷歌开发的，先出现冒泡才出现捕获。

如果一开始不传值的话，就是flase，就是事件冒泡。

如果一开始传值为true，就是事件捕获。

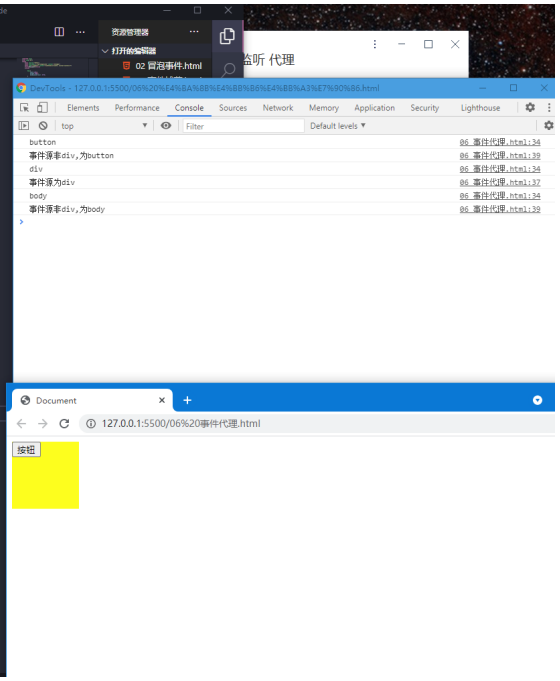
```
/*
    事件捕获:
        false(默认值)    事件冒泡
        true             事件捕获
            先执行事件捕获再执行事件冒泡;

*/
var box = document.getElementById('box');
var p1 = document.getElementById('p1');

//DOM二级:
    //addEventListener( 参数一[事件类型], 参数二【功能】, 参数三【true,false】 )
//可以处理大个事件问题;
box.addEventListener( "click" , function(){
    console.log( "box1" )
},false)

p1.addEventListener( "click" , function(){
    console.log( "p1" )
},false) //F p1 box1 doc
        //T  doc box1 p1

document.addEventListener("click",function(){
    console.log( 'document' )
},false)
```

事件代理

```
        width:100px;
        height:100px;
        background-color: #ff0;
    }

</style>
</head>
<body>


<div id="box">
    <button> 按钮 </button>
</div>


<script>

    var box = document.getElementById("box");

    //ev.target : 触发事件的源头;
    document.onclick = function( ev ){


        console.log( ev.target.tagName.toLowerCase() )

        if( ev.target.tagName.toLowerCase() == "div" ){
            console.log( '事件源为div' )
        }else{
            console.log( '事件源非div,为' + ev.target.tagName.toLowerCase() )
        }


    }



</script>
```